Digital Vector · VCE Applied Computing | Unit 1 · Area of Study 2 · Programming · KK06

Who owns what you make — and what you use ?

Every game, app and website is built from created things : code, artwork, music, fonts, fragments of other people's libraries. The moment you make something original, the law treats it as yours. The moment you reach for someone else's work, the law asks one question — do you have permission? This module is the ruleboo

What this key knowledge covers
This summary distils everything you need for **KK06 — Who owns what you make — and what you use ?** in Programming. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

“Echo Run” — Kai ships a game

Kai is a Year 11 student building **Echo Run**, a neon endless-runner he wants to publish on an app store and sell for $1.99. To finish it, Kai has to make five decisions that have nothing to do with code working — and everything to do with the law:

- 1 the **runner sprite** — draw it, or grab one from another game?
- 2 the **background music** — what can he legally play under a paid game?
- 3 the **jump-physics library** — open-source code has strings attached
- 4 the **title font** — yes, fonts are licensed too
- 5 his **own source code** — what licence should *he* release Echo Run under?

We follow Kai through every fork. By the end you'll be able to make the same calls — and write them up the way a VCAA examiner wants to read them.

“Key legal requirements relating to **intellectual property** and **copyright** when designing and developing software.” The support material anchors this to the **Copyright Act 1968 (Cwlth)** and to **Creative Commons** licences and the restrictions that can still apply to them.

Intellectual property: the big umbrella

When you create something with your mind — a tune, a drawing, an invention, a brand name — that creation is **intellectual property (IP)**. You can't hold it like a phone, but the law still lets you own it, control it and earn from it. “IP” is not one law; it's a **family** of protections, and software touches several of them at once.

Protects original **expression** — code, artwork, music, writing, the look of a UI. **Automatic** and free. This is the heavyweight for software.

Protects **brand identity** — a name, logo or slogan that tells customers who made it. Must be registered to get the ® symbol.

Protects a genuinely new **invention or process** — how something works. Expensive, must be applied for, rare for student software.

Protects the **visual appearance** of a product — its shape or pattern. Occasionally relevant to distinctive device or icon designs.

Copyright protects how an idea is *expressed*; a **patent** protects the *idea / invention itself*. You can copyright the code that sorts a list, but you can't copyright the idea of sorting. For Unit 1 software, copyright does the heavy lifting — so the rest of this module focuses there.

The copyright moment

Here's the part students get wrong on exams: copyright is **automatic**. You don't apply, register or pay. It springs into existence the instant an **original** work is **fixed in a material form** — saved to disk, written down, recorded. Before that moment, an *idea in your head* is not protected. After it, the *expression* is.

Kai is *imagining* the runner sprite. It only exists in his head — a swirl of possibility. Copyright protects nothing yet, because there is no fixed expression to protect.

↑ Toggle the lab. Watch the loose “idea” cloud condense into a solid, sealed work the moment Kai saves the file. The © shield appears **by itself** — no paperwork.

- The sprite PNG Kai drew and saved
- His Echo Run source code on disk
- A level theme he recorded as an MP3
- The wording of his game's description
- “A game where you dodge stuff” — an idea
- A genre, mechanic or rule on its own
- A fact (e.g. high score = 9,000)
- A title or single word (that's trademark territory)

What copyright covers inside a piece of software

A finished app is a **bundle of separately-copyrighted works**. Pull Echo Run apart and almost every layer is owned by someone — usually whoever created it. The default rule: **the creator is the first owner** (unless they made it as an employee, or signed the rights away).

Literary work under the Act. The way Kai wrote his functions and classes is protected — even if the underlying logic is common.

Sprites, backgrounds, icons, button styling. Each is an artistic work with its own copyright.

Music tracks and sound effects are protected. A track usually carries *two* rights — the composition and the recording.

Typeface files are licensed software. “Free to download” does *not* always mean “free to ship in a paid app”.

Tutorial copy, store description, in-game dialogue — written works, all protected.

“I found it on Google Images, so it's free.” **Being able to download something is not permission to use it.** Search results, screenshots and “free” asset packs are full of copyrighted work. Use without a licence = infringement, even if you never charged a cent.

The licence spectrum

A **licence** is permission, with conditions, granted by the owner. Licences sit on a spectrum from **“all rights reserved”** (you may do nothing without asking) to **public domain** (do anything). Slide along it — the further right you go, the more freedom you get, and usually the fewer strings attached.

↑ Drag the slider. The marker glides from the locked red end to the open green end; the permission chips show exactly what each licence lets Kai do with someone else's work.

The licence types you must know

For the exam, you need to recognise the main families of licence and what each one signals about cost and freedom.

A **royalty-free** asset means you pay *once* and owe no per-use fee afterwards — you might still pay an upfront price and still must follow the licence. Don't confuse it with “free of charge”.

Creative Commons, decoded

Creative Commons (CC) lets a creator keep copyright but **pre-grant permission** on their own terms. A CC licence is built from up to four **condition blocks**. Read the blocks and you instantly know what you're allowed to do — this is exactly the “restrictions that can still apply” the study design wants you to understand.

Credit the creator. Present in *every* CC licence.

If you remix it, release your version under the *same* licence.

No making money from it. **Kills a paid game.**

Use as-is. No editing, cropping or remixing.

Kai finds a perfect track licensed **CC BY-NC**. Decode it: **BY** = he must credit the artist; **NC** = non-commercial only. But Kai wants to **sell** Echo Run for $1.99 — that's commercial use. **Verdict: he can't use it.** He needs a track that is CC BY (commercial OK), CC0, royalty-free with a commercial licence, or one he makes himself.

Assuming “Creative Commons” means “free for anything”. It doesn't. **NC** blocks commercial use and **ND** blocks editing. Always read the suffix before you build a paid product on top of it.

Permissive vs copyleft — the chain reaction

Open-source code is free to use, but **the licence on a library can reach into your whole project**. This is the difference between **permissive** licences (like MIT) and **copyleft** licences (like the GPL). For Echo Run, Kai needs a jump-physics library — and his choice decides whether he can keep his own code private.

↑ “Add GPL library” triggers a copyleft wave: the obligation spreads up through Kai's stack until **everything** turns copyleft-purple. MIT leaves his cubes blue and free.

A developer who drops a GPL library into a closed-source paid product and ships it has **breached the licence** — a legal problem, not just an ethical one. Knowing the difference

between permissive and copyleft is a genuine professional skill, and a favourite VCAA distinguishing question.

A sprite ripped from a popular console game

A music track licensed CC BY (for a paid game)

A music track licensed CC BY-NC (for a paid game)

An MIT-licensed jump-physics library

A sprite Kai drew himself in Aseprite

Infringement, plagiarism & consequences

Two words students mix up — and an examiner loves to test the difference.

Using a protected work **without permission** — copying, sharing, performing or adapting it. This is a breach of the **law**. It happens even if you *credit* the creator, because crediting is not the same as having a licence.

Passing off someone else's work as **your own** — failing to credit. This is mainly an **ethical / academic** wrong (and against school rules). You can plagiarise something that is *not* even under copyright.

You can **infringe without plagiarising** (use a licensed song in a video and credit it, but never get the licence) and **plagiarise without infringing** (copy a public-domain poem and claim you wrote it). Different problems, different fixes: a *licence* fixes infringement; *attribution* fixes plagiarism.

What actually happens when you get it wrong

The app store or platform removes your product, often without warning.

Repeat strikes can close your developer or channel account entirely.

The owner can demand damages or an account of profits. Legal costs dwarf any sales.

“The dev who stole assets” follows you. In a small industry, that matters.

Kai's release checklist

Here's how Kai legally ships Echo Run. This is the practical workflow examiners want you to be able to recommend.

- **Make or license every asset.** Original art & code, or assets whose licence allows commercial use. Keep proof.
- **Read the full licence, not the headline.** Decode CC suffixes; check MIT vs GPL on every library.
- **Attribute where required.** A credits screen listing CC BY assets, libraries and fonts.
- **Keep your own licence notices intact.** Don't strip the MIT notice from a library you reused.
- **Check the font.** Confirm the font licence permits embedding in a commercial app.
- **Choose a licence for your code.** Proprietary to keep it closed and sell it, or open source to share it.
- **Mark your ownership.** Add a copyright notice (© 2026 Kai) — not required for protection, but good practice.

Tick them off as you read — your progress saves automatically.

Exam questions — with weak & strong responses

These are written in VCAA style. For each: read the command word, attempt your own answer, then reveal a **weak** response, a **strong** response and the **marking guide**. Use the self-mark dial to bank what you'd score yourself.

Define the term **intellectual property** and give one example relevant to developing software.

Intellectual property is stuff you own on a computer like files and programs.

Intellectual property is a creation of the mind — such as a work, invention or brand — that the law lets a person own and control. Example: the original source code a developer writes for a game is their intellectual property, protected by copyright.

- 1 mark — a correct definition: a creation of the mind that can be legally owned/controlled.
- 1 mark — a relevant software example (e.g. original source code, game artwork, a track).

Explain how the Copyright Act 1968 (Cwlth) protects a student's original game code, including when that protection begins and what it covers.

Copyright protects your work after you register it with the government so no one can copy your code.

Under the Copyright Act 1968 (Cwlth), original source code is protected as a literary work. Protection is automatic and free — it begins the moment the code is fixed in material form (e.g. saved to disk), with no registration required. It protects the student's particular expression of the code, not the underlying idea or algorithm, so others may not copy or adapt that code without permission.

The P·E·L writing trainer

Most marks are lost not because students don't know the content, but because they don't **structure** the answer. For “explain” and “justify” questions, write in **P·E·L**: **Point** · **Explain** · **Link**. Build one below.

P Kai cannot legally use the CC BY-NC track in Echo Run. E The “NC” block means the licence permits non-commercial use only; because Kai intends to sell the game for $1.99, his use is commercial and falls outside what the licence grants. L Using it anyway would be copyright infringement — exposing him to a store takedown — so he must source a track licensed for commercial use or create his own.

Command-word decoder

The verb in the question tells you how much to write. Match the command word to the move — and the marks usually match the depth.

Give a fact or name — no explanation needed.

Give the precise meaning of a term.

Give the features or characteristics — what it is and looks like.

Say how or why — show cause, effect and reasoning.

Show the difference(s) between two things, point by point.

Make a choice and defend it with reasons that beat the alternatives.

Key-term flip cards

1 When does copyright protection of original software begin in Australia?

2 A track is licensed CC BY-NC. Kai wants to sell his game. Can he use it?

3 What is the key obligation a GPL (copyleft) library places on a derivative work?

4 Which best distinguishes infringement from plagiarism?

5 Copyright protects ___, while a patent protects ___.

6 A font is 'free to download' but states no licence. The safest action is to:

One-page cheat sheet

- **IP** = copyright + trademark + patent + design.
- **Copyright** is automatic, free, protects *expression fixed in material form*.
- **Copyright Act 1968 (Cwlth)** is the law.
- **Licence** = permission with conditions from the owner.
- **Creator is first owner** by default.
- © All rights reserved → ask first.
- Proprietary / EULA → buy a use licence.
- Freeware / shareware → free use, closed source.
- Open source → MIT (permissive) vs GPL (copyleft).
- CC0 / public domain → do anything.
- **BY** — credit the creator (always).
- **SA** — share remixes under the same licence.
- **NC** — no commercial use (kills a paid app).
- **ND** — no edits or remixes.
- **Infringement** = no licence (legal). Fix: get a licence.
- **Plagiarism** = no credit (ethical). Fix: attribute.
- **Copyright** = expression. **Patent** = invention.
- **Royalty-free** ≠ free of charge.
- **Download ≠ permission.**

That's KK06. You can now define IP, explain how copyright protects software, decode any licence, and recommend a legal release path — and write it up in P·E·L.

Common traps & misconceptions

Exam trap
1 **“Copyright needs to be registered.”** Wrong in Australia. Copyright is automatic on fixation. Registration belongs to *trademarks* and *patents*.

Exam trap
2 **“I credited them, so it's fine.”** Attribution fixes plagiarism, not infringement. You still need a *licence* to use protected work.

Exam trap
3 **“Creative Commons = free for anything.”** NC blocks selling; ND blocks editing. Always decode the suffix.

Exam trap
4 **“Open source means no rules.”** Open source still has a licence. GPL (copyleft) can force your whole project open.

Exam trap
5 **Confusing copyright with patent.** Copyright = the expression (the code as written). Patent = the invention/idea (a novel process).

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Define

The term **intellectual property** and give one example relevant to developing software.

✗ A weak answer
Intellectual property is stuff you own on a computer like files and programs. Too vague and wrong — IP isn't 'files on a computer'. No clear example tied to creation of the mind.

✓ A strong answer
Intellectual property is a creation of the mind — such as a work, invention or brand — that the law lets a person own and control. Example: the original source code a developer writes for a game is their intellectual property, protected by copyright.

How the marks are awarded

- 1 mark — a correct definition: a creation of the mind that can be legally owned/controlled.
- 1 mark — a relevant software example (e.g. original source code, game artwork, a track).

Q2. Explain

Explain how the Copyright Act 1968 (Cwlth) protects a student's original game code, including when that protection begins and what it covers.

✗ A weak answer
Copyright protects your work after you register it with the government so no one can copy your code. Factually wrong — there is no registration for copyright in Australia. Misses 'expression vs idea'.

✓ A strong answer
Under the Copyright Act 1968 (Cwlth), original source code is protected as a literary work. Protection is automatic and free — it begins the moment the code is fixed in material form (e.g. saved to disk), with no registration required. It protects the student's particular expression of the code, not the underlying idea or algorithm, so others may not copy or adapt that code without permission.

How the marks are awarded

- 1 mark — protection is automatic, no registration needed.
- 1 mark — begins when the work is fixed in material form.
- 1 mark — protects the expression (the code as written), not the idea.

Q3. Explain

A developer wants to use a music track licensed under **CC BY-NC** in a game they intend to sell. **Explain two** conditions this licence places on the developer and the consequence for this project.

✗ A weak answer
They have to say thanks to the person and it's free so they can use it in the game. Identifies attribution loosely but misses NC entirely and reaches the wrong conclusion for a paid game.

✓ A strong answer
The 'BY' condition requires the developer to attribute (credit) the original creator wherever the track is used. The 'NC' condition restricts use to non-commercial purposes only. Because the developer intends to sell the game, their use is commercial and therefore falls outside what the licence permits — so they cannot legally use this track and would need a commercially-licensed or self-made alternative.

How the marks are awarded

- 1 mark — BY = attribution required.
- 1 mark — NC = non-commercial use only.
- 1 mark — selling the game is commercial use.
- 1 mark — therefore the track cannot be used / licence would be breached.

Q4. Distinguish

Distinguish between a permissive open-source licence (e.g. MIT) and a copyleft licence (e.g. GPL), with reference to the obligations each places on a derivative work.

✗ A weak answer

MIT and GPL are both free open source licences so you can use them in any project without rules. States they're 'the same / no rules' — the exact misconception the question tests. Zero distinction made.

✓ A strong answer

Both publish source code for reuse, but they differ in the obligations on derivative works. A permissive licence such as MIT lets a developer reuse the code, keep their own code closed/private and release the larger project under any licence they choose, provided they retain the original notice. A copyleft licence such as the GPL requires that any derivative work which includes the GPL code must also be released as open source under the same (GPL) licence — so a developer cannot keep their project closed if they build on GPL code.

How the marks are awarded

- 1 mark — both make source available for reuse (shared trait).
- 1 mark — MIT: permissive, code can stay closed / any licence (retain notice).
- 1 mark — GPL: copyleft, derivative must be open source.
- 1 mark — under the same/GPL licence (clear contrast on derivative obligation).

Revision checklist

- [ ] Make or license every asset. Original art & code, or assets whose licence allows commercial use. Keep proof.
- [ ] Read the full licence, not the headline. Decode CC suffixes; check MIT vs GPL on every library.
- [ ] Attribute where required. A credits screen listing CC BY assets, libraries and fonts.
- [ ] Keep your own licence notices intact. Don't strip the MIT notice from a library you reused.
- [ ] Check the font. Confirm the font licence permits embedding in a commercial app.
- [ ] Choose a licence for your code. Proprietary to keep it closed and sell it, or open source to share it.
- [ ] Mark your ownership. Add a copyright notice (© 2026 Kai) — not required for protection, but good practice.